



Marwadi
University
Marwadi Chandarana Group



MARWADI UNIVERSITY
FACULTY OF COMPUTER APPLICATIONS
PYTHON PROGRAMMING (05CS2104) – LAB MANUAL

Python Programming (05CS2104)

Lab Manual (2025-26)

Program: M.Sc. (Cyber Security & Cyber Law)

Semester: 1

Name of the Student: Vala Lakshaydeepsinh

Enrolment No.: 92600565003

INDEX

Lab	Program	Date	Remarks	Signature
1.	Python Program to Print Hello World.			
2.	Python program to do arithmetical operations			
3.	Python program to swap two variables			
4.	Python Program to Find the Factorial of a Number			
5.	Python Program to Print the Fibonacci sequence			
6.	Python program to print the elements of an array			
7.	Python program to print the largest element and smallest element in an array			
8.	Python program to add two matrix using array and function			
9.	Python Program to concatenate two strings			
10.	Python Programs to perform various operations on Strings using functions			
11.	Python Programs to demonstrate use of list and various functions of it			
12.	Python Programs to demonstrate use of tuple and various functions of it			
13.	Python Programs to demonstrate the use of dictionary and various functions of it			
14.	Python Programs to create a function (Make your own assumptions)			
15.	Python Programs to demonstrate use of various arguments which can be passed to functions			
16.	Python Program to demonstrate the use of class			
17.	Python Program to demonstrate the concept of inner class			
18.	Python program to demonstrate the use of methods			
19.	Python program to demonstrate the use of methods			
20.	Write a Python program to show method overloading by adding two numbers and three numbers using the same method name.			
21.	Write a Python program to show method overriding using a parent class Animal and a child class Dog.			
22.	Python programs to demonstrate the use of regular expression			

23.	Python programs to demonstrate the use of database using mysql			
24.	Python Program to demonstrate the use of DataFrames in python			
25.	Python Programs to demonstrate different ways of creating DataFrames			
26.	Python Programs to create Pie chart			
27.	Python Programs to create histogram			
28.	Python Programs to create bar graph			
29.	Python Programs to create Line graph			
30.	Python Programs to work with Python Libraries useful for Cyber security. Program to generate hash values of a string using hashlib library.			
31.	Python Program using socket — Simple Port Scanner			
32.	Python Program using re — Detect Emails Using Regular Expressions			
33.	Python Program using os Library — Check Files in a Directory			
34.	Python program for whois lookup			
35.	Python program for Ping Scanner shows ip up or down			

Practical No. 1

Objective: Python Program to Print Hello World.

Source Code:

```
print("M.sc. (CS&CL)Semester 1")  
print("Enrollment No:92600565003")  
print("Hello Word")
```

Output:

```
===== RESTART: D:\LV\Msc_92600565003\1.py =====  
M.sc. (CS&CL)Semester 1  
Enrollment No:92600565003  
Hello Word
```

Conclusion: The Python program to display output text using the print() function was successfully executed.

Practical No. 2

Objective: Python program to do arithmetical operations

Source Code:

```
print("M.sc. (CS&CL)Semester 1")  
print("Enrollment No:92600565003")
```

```
a =float(input("enter 1st num:"))  
b =float(input("enter 2nd num:"))
```

```
print("Addition = ",a+b)  
print("Substraction = ",a-b)  
print("Multiplication = ",a*b)  
print("Division = ",a/b)  
print("Floor Division = ",a//b)  
print("Modulus = ",a%b)  
print("Exponentiation = ",a**b)
```

Output:

```
===== RESTART: D:\LV\Msc_92600565003\2.py ==  
M.sc. (CS&CL)Semester 1  
Enrollment No:92600565003  
enter 1st num:12  
enter 2nd num:15  
Addition = 27.0  
Substraction = -3.0  
Multiplication = 180.0  
Division = 0.8  
Floor Division = 0.0  
Modulus = 12.0  
Exponentiation = 1.5407021574586368e+16  
>
```

Conclusion: Basic arithmetic operations (addition, subtraction, multiplication, division) were successfully implemented and calculated using standard Python operators.

Practical No. 3

Objective: Python program to swap two variables

Source Code:

```
print("M.sc. (CS&CL)Semester 1")
print("Enrollment No:92600565003")

a =float(input("enter 1st num:"))
b =float(input("enter 2nd num:"))

print("before swap")
print("a=",a)
print("b=",b)

a,b=b,a

print("\nafter swap")
print("a=",a)
print("b=",b)
```

Output:

```
===== RESTART: D:\LV\Msc_92600565003\3.py
M.sc. (CS&CL)Semester 1
Enrollment No:92600565003
enter 1st num:15
enter 2nd num:12
before swap
a= 15.0
b= 12.0

after swap
a= 12.0
b= 15.0
```

Conclusion: The values of two variables were successfully swapped without requiring a temporary variable using Python tuple unpacking.

Practical No. 4

Objective: Python Program to Find the Factorial of a Number

Source Code:

```
print("M.sc. (CS&CL)Semester 1")
print("Enrollment No:92600565003")
num = int(input('enter a num'))
factorial=1

if num<0:
    print("factorial is not defined")

else:
    for i in range (1,num+1):
        factorial*=i
    print("factorial of",num,"is",factorial)
```

Output:

```
===== RESTART: D:\LV\Msc_92600565003\6.py =====
M.sc. (CS&CL)Semester 1
Enrollment No:92600565003
enter a num15
factorial of 15 is 1307674368000
```

Conclusion: The factorial of a given integer was successfully computed using an iterative loop structure..

Practical No. 5

Objective: Python Program to Print the Fibonacci sequence

Source Code:

```
print("M.sc. (CS&CL)Semester 1")
print("Enrollment No:92600565003")

n = int(input("Enter the number: "))

a, b = 0, 1

print("Fibonacci Sequence:")

for i in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

Output:

```
===== RESTART: D:\LV\Msc_92600565003\7.py
M.sc. (CS&CL)Semester 1
Enrollment No:92600565003
Enter the number: 12
Fibonacci Sequence:
0 1 1 2 3 5 8 13 21 34 55 89
```

Conclusion: The Fibonacci sequence up to the specified number of terms was successfully generated and printed.

Practical No. 6

Objective: Python program to print the elements of an array

Source Code:

```
print("M.sc. (CS&CL)Semester 1")
print("Enrollment No:92600565003")
arr = [10, 20, 30, 40, 50]
print("Elements of array are:")
for i in range(len(arr)):
    print(arr[i])
```

Output:

```
===== RESTART: D:\LV\Msc_92600565003\8.py =====
M.sc. (CS&CL)Semester 1
Enrollment No:92600565003
Elements of array are:
10
20
30
40
50
```

Conclusion: Array (list) elements were successfully iterated through and displayed on the screen using a loop.

Practical No. 7

Objective: Python program to print the largest element and smallest element in an array

Source Code:

```
print("M.sc. (CS&CL)Semester 1")
print("Enrollment No:92600565003")
arr = [15, 41, 77, 95, 12]

largest = arr[0]
smallest = arr[0]

for i in range(1, len(arr)):
    if arr[i] > largest:
        largest = arr[i]
    if arr[i] < smallest:
        smallest = arr[i]

print("Largest element:", largest)
print("Smallest element:", smallest)
```

Output:

```
===== RESTART: D:\LV\Msc_92600565003\9.py =====
M.sc. (CS&CL)Semester 1
Enrollment No:92600565003
Largest element: 95
Smallest element: 12
```

Conclusion: The largest and smallest elements within an array were successfully identified and displayed using built-in functions.

Practical No. 8

Objective: Python program to add two matrix using array and function

Source Code:

```
print("M.sc. (CS&CL)Semester 1")
print("Enrollment No:92600565003")
A = [[1, 2, 3],
      [4, 5, 6],
      [7, 8, 9]]
B = [[9, 8, 7],
      [6, 5, 4],
      [3, 2, 1]]
result = [[0, 0, 0],
           [0, 0, 0],
           [0, 0, 0]]

for i in range(len(A)):
    for j in range(len(A[0])):
        result[i][j] = A[i][j] + B[i][j]
print("Resultant Matrix after addition:")
for row in result:
    print(row)
```

Output:

```
===== RESTART: D:\LV\Msc_92600565003\10.py
M.sc. (CS&CL)Semester 1
Enrollment No:92600565003
Resultant Matrix after addition:
[10, 10, 10]
[10, 10, 10]
[10, 10, 10]
```

Conclusion: Matrix addition was successfully performed element-wise using nested arrays and a custom function.

Practical No. 9

Objective: Python Program to concatenate two strings

Source Code:

```
print("M.sc. (CS&CL)Semester 1")  
print("Enrollment No:92600565003")  
str1 = "Hello"  
str2 = "World"  
print(str1 + " " + str2)
```

Output:

```
===== RESTART: D:/LV/Msc_92600565003/11.py  
M.sc. (CS&CL)Semester 1  
Enrollment No:92600565003  
Hello World
```

Conclusion: Two string variables were successfully joined into a single string using the concatenation operator.

Practical No. 10

Objective: Python Programs to perform various operations on Strings using functions

Source Code:

```
print("M.sc. (CS&CL)Semester 1")
print("Enrollment No:92600565003")
s = "Python Programming"
print(s.upper())
print(s.lower())
print(s.replace("Python", "Java"))
print(s.split())
```

Output:

```
===== RESTART: D:/LV/Msc_92600565003/12.py
M.sc. (CS&CL)Semester 1
Enrollment No:92600565003
PYTHON PROGRAMMING
python programming
Java Programming
['Python', 'Programming']
```

Conclusion: Various string manipulation operations were successfully executed using standard built-in string functions.

Practical No. 11

Objective: Python Programs to demonstrate use of list and various functions of it

Source Code:

```
print("M.sc. (CS&CL)Semester 1")
print("Enrollment No:92600565003")
# 1. CREATING LISTS
```

```
numbers = [10, 20, 30, 40]
mixed = ['Python', 100, 3.14, True]
print("Numbers list:", numbers)
print("Mixed list:", mixed)
```

2. ACCESSING LIST ELEMENTS

```
fruits = ['apple', 'banana', 'cherry', 'date']

print("First element (index 0):", fruits[0])
print("Last element (index -1):", fruits[-1])
```

3. UPDATING LISTS

```
items = [10, 20, 30, 40]

items[1] = 99      # Update single element
items[2:4] = [77, 88] # Update slice range

print("Updated list:", items)
```

4. DELETE LIST ELEMENTS

```
del_items = ['a', 'b', 'c', 'd']
del del_items[1]    # Delete element at index 1 ('b')

print("After del items[1]:", del_items)
```

5. BASIC LIST OPERATIONS

```
list_a = [1, 2]
list_b = [3, 4]

print("Length len():", len(list_a))
```

```
print("Concatenation (+):", list_a + list_b)
print("Repetition (*):", list_a * 3)
print("Membership (in):", 2 in list_a)
```

6. INDEXING, SLICING AND MATRICES

```
tech = ['C++', 'Java', 'Python', 'Go', 'Rust']
matrix = [
    [1, 2, 3],
    [4, 5, 6]
]
print("Offset index [2]:", tech[2])
print("Negative index [-2]:", tech[-2])
print("Slicing [1:4]:", tech[1:4])
print("Matrix row 0, col 1:", matrix[0][1])
```

7. BUILT-IN FUNCTIONS

```
nums_func = [45, 12, 89, 3]
sample_tuple = (1, 2, 3)

print("len():", len(nums_func))
print("max():", max(nums_func))
print("min():", min(nums_func))
print("list(tuple):", list(sample_tuple))
```

8. BUILT-IN METHODS

```
# append()
data = [1, 2]
data.append(3)
print("append(3):", data)

# count()
count_data = [10, 20, 10, 30, 10]
print("count(10):", count_data.count(10))

# extend()
ext_data = [1, 2]
ext_data.extend([3, 4])
print("extend([3, 4]):", ext_data)

# index()
idx_data = ['red', 'green', 'blue']
print("index('green'):", idx_data.index('green'))

# insert()
ins_data = [10, 30]
```

```
ins_data.insert(1, 20)
print("insert(1, 20):", ins_data)

# pop()
pop_data = [10, 20, 30]
popped_val = pop_data.pop()
print("pop value:", popped_val)
print("after pop():", pop_data)

# remove()
rem_data = ['apple', 'banana', 'cherry']
rem_data.remove('banana')
print("remove('banana'):", rem_data)

# reverse()
rev_data = [1, 2, 3, 4]
rev_data.reverse()
print("reverse():", rev_data)

# sort()
sort_data = [40, 10, 30, 20]
sort_data.sort()
print("sort() ascending:", sort_data)

sort_data.sort(reverse=True)
print("sort() descending:", sort_data)
```

Output:

```
===== RESTART: D:\LV\Msc_92600565003\unit2\1.py
M.sc. (CS&CL)Semester 1
Enrollment No:92600565003
Numbers list: [10, 20, 30, 40]
Mixed list: ['Python', 100, 3.14, True]
First element (index 0): apple
Last element (index -1): date
Updated list: [10, 99, 77, 88]
After del items[1]: ['a', 'c', 'd']
Length len(): 2
Concatenation (+): [1, 2, 3, 4]
Repetition (*): [1, 2, 1, 2, 1, 2]
Membership (in): True
Offset index [2]: Python
Negative index [-2]: Go
Slicing [1:4]: ['Java', 'Python', 'Go']
Matrix row 0, col 1: 2
len(): 4
max(): 89
min(): 3
list(tuple): [1, 2, 3]
append(3): [1, 2, 3]
count(10): 3
extend([3, 4]): [1, 2, 3, 4]
index('green'): 1
insert(1, 20): [10, 20, 30]
pop value: 30
after pop(): [10, 20]
remove('banana'): ['apple', 'cherry']
reverse(): [4, 3, 2, 1]
sort() ascending: [10, 20, 30, 40]
sort() descending: [40, 30, 20, 10]
```

Conclusion:

In this practical, various list operations such as adding, removing, and sorting elements were successfully implemented using built-in methods like `append()`, `pop()`, and `sort()`.

Practical No. 12

Objective: Python Programs to demonstrate use of tuple and various functions of it

Source Code:

```
print("M.sc. (CS&CL)Semester 1")
print("Enrollment No:92600565003")
#Accessing Values in Tuples
tup1 = ('physics', 'chemistry', 1997, 2000)
tup2 = (1, 2, 3, 4, 5, 6, 7)
print ("tup1[0]: ", tup1[0])
print ("tup2[1:5]: ", tup2[1:5])
```

#Updating Tuples

```
tup1 = (12, 34.56)
tup2 = ('abc', 'xyz')
tup3 = tup1 + tup2
print (tup3)
```

#Delete Tuple Elements

```
tup= ('physics', 'chemistry', 1997, 2000);
print (tup)
del tup1;
print ("After deleting tup : ")
print (tup)
```

#Basic Tuples Operation

1. Length

```
print(len((1, 2, 3)))
```

2. Concatenation

```
print((1, 2, 3) + (4, 5, 6))
```

3. Repetition

```
print(("Hi!",) * 4)
```

4. Membership

```
print(3 in (1, 2, 3))
```

5. Iteration

```
for x in (1, 2, 3):
    print(x, end=" ")

# Sample Tuple & Matrix
T = ('C++', 'Java', 'Python')
matrix = (
    (1, 2, 3),
    (4, 5, 6),
    (7, 8, 9)
)

# 1. Indexing
print(T[2])
print(T[-2])

# 2. Slicing
print(T[1:])
print(T[:2])
print(T[::-1])

# 3. Matrix Access
print(matrix[0])
print(matrix[1][2])
```

Output:

```
===== RESTART: D:\LV\Msc_92600565003\unit2\2.py
M.sc. (CS&CL)Semester 1
Enrollment No:92600565003
tup1[0]:  physics
tup2[1:5]:  (2, 3, 4, 5)
(12, 34.56, 'abc', 'xyz')
('physics', 'chemistry', 1997, 2000)
After deleting tup :
('physics', 'chemistry', 1997, 2000)
3
(1, 2, 3, 4, 5, 6)
('Hi!', 'Hi!', 'Hi!', 'Hi!')
True
1 2 3 Python
Java
('Java', 'Python')
('C++', 'Java')
('Python', 'Java', 'C++')
(1, 2, 3)
6
```

Conclusion:

This practical demonstrated the immutability of Python tuples and how to perform operations like indexing, slicing, and accessing tuple elements using methods like count() and index().

Practical No. 13

Objective: Python Programs to demonstrate the use of dictionary and various functions of it

Source Code:

```
print("M.sc. (CS&CL)Semester 1")
print("Enrollment No:92600565003")
def demonstrate_dictionary_operations():
    print(f"Student Enrolment No: 92600565003\n")

    # 1. Creating a Dictionary
    student = {
        "name": "lakshyadeepsinh",
        "age": 21,
        "course": "M.Sc. CS&CL",
        "marks": 85
    }
    print("Initial Dictionary:", student)

    # 2. Accessing Values
    print("Name:", student["name"])
    print("Course (using get()):", student.get("course"))
    print("Non-existent Key (with default):", student.get("gpa", "Not Available"))

    # 3. Adding and Updating Elements
    student["age"] = 22 # Direct assignment
    student.update({"city": "Vadodara", "marks": 90}) # Batch update
    print("After Update:", student)

    # 4. Built-in Retrieval Functions
    print("Keys:", list(student.keys()))
    print("Values:", list(student.values()))
    print("Items (Key-Value pairs):", list(student.items()))

    # 5. Removing Elements
    removed_val = student.pop("marks")
    print(f"Popped 'marks': {removed_val}")

    last_item = student.popitem() # Removes last inserted item
    print(f"Popped last item: {last_item}")

    del student["age"]
    print("After deleting 'age':", student)
```

```
# 6. Copying and Clearing
temp_dict = student.copy()
temp_dict.clear()
print("Cleared Dictionary:", temp_dict)

if __name__ == "__main__":
    demonstrate_dictionary_operations()
```

Output:

```
===== RESTART: D:/LV/Msc_92600565003/unit2/3.1.py =====
M.sc. (CS&CL)Semester 1
Enrollment No:92600565003
Student Enrolment No: 92600565003

Initial Dictionary: {'name': 'lakshyadeepsinh', 'age': 21, 'course': 'M.Sc. CS&CL', 'marks': 85}
Name: lakshyadeepsinh
Course (using get()): M.Sc. CS&CL
Non-existent Key (with default): Not Available
After Update: {'name': 'lakshyadeepsinh', 'age': 22, 'course': 'M.Sc. CS&CL', 'marks': 90, 'city': 'Vadodara'}
Keys: ['name', 'age', 'course', 'marks', 'city']
Values: ['lakshyadeepsinh', 22, 'M.Sc. CS&CL', 90, 'Vadodara']
Items (Key-Value pairs): [('name', 'lakshyadeepsinh'), ('age', 22), ('course', 'M.Sc. CS&CL'), ('marks', 90), ('city', 'Vadodara')]
Popped 'marks': 90
Popped last item: ('city', 'Vadodara')
After deleting 'age': {'name': 'lakshyadeepsinh', 'course': 'M.Sc. CS&CL'}
Cleared Dictionary: {}
```

Conclusion:

In this practical, key-value data manipulation was implemented successfully using Python dictionaries and built-in methods such as keys(), values(), items(), and get().

Practical No. 14

Objective: Python Programs to create a function (Make your own assumptions)

Source Code:

```
print("M.sc. (CS&CL)Semester 1")
print("Enrollment No:92600565003")

def area_of_circle(radius):
    return 3.14159 * (radius ** 2)

def area_of_triangle(base, height):
    return 0.5 * base * height
```

```
def area_of_square(side):  
    return side ** 2  
  
def area_of_rectangle(length, width):  
    return length * width  
  
def area_of_trapezoid(base1, base2, height):  
    return 0.5 * (base1 + base2) * height  
  
def interest(principal, rate, time):  
    return (principal * rate * time) / 100  
  
print("Circle Area (r=7):", area_of_circle(7))  
print("Triangle Area (b=8, h=5):", area_of_triangle(8, 5))  
print("Square Area (s=6):", area_of_square(6))  
print("Area of Rectangle:", area_of_rectangle(10, 5))  
print("Area of Trapezoid:", area_of_trapezoid(6, 10, 4))  
print("Interest:", interest(1000, 5, 2))
```

Output:

```
===== RESTART: D:\LV\Msc_92600565003\unit2\4.py  
M.sc. (CS&CL)Semester 1  
Enrollment No:92600565003  
Circle Area (r=7): 153.93791  
Triangle Area (b=8, h=5): 20.0  
Square Area (s=6): 36  
Area of Rectangle: 50  
Area of Trapezoid: 32.0  
Interest: 100.0
```

Conclusion:

This practical demonstrated the modular coding approach in Python by defining custom functions using the def keyword, executing logic, and returning values to the caller.

Practical No. 15

Objective: Python Programs to demonstrate use of various arguments which can be passed to functions

Source Code:

```
print("M.sc. (CS&CL)Semester 1")  
print("Enrollment No:92600565003")
```

1. Positional Arguments

```
def area(w, h):  
    return w * h
```

2. Default Arguments

```
def greet(name, msg="Hello"):  
    print(f'{msg}, {name}!')
```

3. Keyword Arguments

```
def student_info(name, age):  
    print(f'Name: {name}, Age: {age}')
```

4. Variable-length Positional Arguments (*args)

```
def add_all(*numbers):  
    print("Sum:", sum(numbers))
```

5. Variable-length Keyword Arguments (**kwargs)

```
def print_details(**details):  
    for key, value in details.items():  
        print(f'{key}: {value}')
```

Main Execution

```
print(" Student Enrolment No: 92600565003 ")  
print(" Name: Lakshyadeepsinh\n")
```

Demonstrating function calls

```
print("Area:", area(5, 10))  
greet("Lakshyadeepsinh")  
student_info(age=22, name="Lakshyadeepsinh")  
add_all(10, 20, 30)  
print_details(Role="Student", City="Rajkot")
```

Output:

```
===== RESTART: D:/LV/Msc_92600565003/unit2/15.py  
M.sc. (CS&CL) Semester 1  
Enrollment No:92600565003  
Student Enrolment No: 92600565003  
Name: Lakshyadeepsinh  
  
Area: 50  
Hello, Lakshyadeepsinh!  
Name: Lakshyadeepsinh, Age: 22  
Sum: 60  
Role: Student  
City: Rajkot
```

Conclusion:

In this practical, different types of function parameters—including positional, keyword, default, and arbitrary arguments (*args and **kwargs)—were successfully executed and analyzed.

Practical No. 16

Objective: Python Program to demonstrate the use of class

Source Code:

```
print("M.sc. (CS&CL)Semester 1")
print("Enrollment No:92600565003")
#class&objects
class MyNewClass:
    """This class demonstrates the creation of objects"""
    num=100
    def hello(self):
        print("hello word")
obj=MyNewClass()
print(obj.num)
obj.hello()
print(MyNewClass.__doc__)
```

```
class Employee:
    def __init__(self,name,id):
        self.id=id
        self.name=name

    def display(self):
        print(self.id,self.name)
```

```
emp1=Employee("john",101)
emp2=Employee("David",102)
```

```
emp1.display()
emp2.display()
```

```
class Student:
    def __init__(self):
        print("this is non parametrized constructor")
    def show(self,name):
        print("hello",name)
```

```
obj=Student()  
obj.show("Lakshyadeepsinh")
```

Output:

```
===== RESTART: D:\LV\Msc_92600565003\unit2\6.py  
M.sc. (CS&CL)Semester 1  
Enrollment No:92600565003  
100  
hello word  
This class demonstrates the creation of objects  
101 john  
102 David  
this is non parametrized constructor  
hello Lakshyadeepsinh
```

Conclusion:

This practical demonstrated basic Object-Oriented Programming (OOP) by creating a class, instantiating objects, and managing object state through class attributes and instance attributes.

Practical No. 17

Objective: Python Program to demonstrate the concept of inner class

Source Code:

```
print("M.sc. (CS&CL)Semester 1")  
print("Enrollment No:92600565003")  
  
class University:  
    def __init__(self,university_name):  
        self.university_name = university_name  
  
class Student:  
    def __init__(self,name,enrollment_no,department):  
        self.name = name  
        self.enrollment_no = enrollment_no  
        self.department = department  
  
    def display(self):  
        print("Student Name:",self.name)  
        print("Entollment_No:",self.enrollment_no)
```

```
print("Department:",self.department)

univ = University("Marwadi University")
stud = Student("Lakshyadeepsinh Vala",92600565003,"FoCa")

print("University:",univ.university_name)
stud.display()
```

Output:

```
===== RESTART: D:\LV\Msc_92600565003\unit2\8.py =====
M.sc. (CS&CL) Semester 1
Enrollment No:92600565003
University: Marwadi University
Student Name: Lakshyadeepsinh Vala
Entollment_No: 92600565003
Department: FoCa
|
```

Conclusion:

In this practical, the concept of nested classes was implemented. The inner class was successfully instantiated within the outer class to logically group related functionalities.

Practical No. 18

Objective: Python program to demonstrate the use of methods

Source Code:

```
print("M.sc. (CS&CL)Semester 1")
print("Enrollment No:92600565003")

class Student:
    # Constructor
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    # Method 1: Display student details
    def display_info(self):
        print(f"Student Name: {self.name}")
        print(f"Marks: {self.marks}")

    # Method 2: Calculate grade based on marks
    def get_grade(self):
        if self.marks >= 80:
            return "A"
        elif self.marks >= 60:
```

```
        return "B"
    else:
        return "C"
```

Main Execution

```
print("Student Enrolment No: 92600565003")
print("Name: Lakshyadeepsinh \n")
```

Creating an instance of the class

```
student1 = Student("Lakshyadeepsinh", 85)
```

Calling instance methods

```
student1.display_info()
print("Grade:", student1.get_grade())
```

Output:

```
===== RESTART: D:\LV\Msc_92600565003\unit2\9.py =====
M.sc. (CS&CL)Semester 1
Enrollment No:92600565003
Student Enrolment No: 92600565003
Name: Lakshyadeepsinh

Student Name: Lakshyadeepsinh
Marks: 85
Grade: A
```

Conclusion:

This practical demonstrated how instance methods access and modify object state using the self reference during program execution.

Practical No. 19

Objective: Python program to demonstrate various types of methods

Source Code:

```
print("M.sc. (CS&CL)Semester 1")
print("Enrollment No:92600565003")

class Student:
    # Class Variable (Shared across all instances)
    college_name = "Marwadi University"

    def __init__(self, name, marks):
        # Instance Variables
```

```
self.name = name
self.marks = marks
```

```
# 1. Instance Method (Works with instance attributes using 'self')
```

```
def display_details(self):
    print(f"Student Name: {self.name}")
    print(f"Marks: {self.marks}")
```

```
# 2. Class Method (Works with class attributes using 'cls')
```

```
@classmethod
def get_college(cls):
    print(f"College Name: {cls.college_name}")
```

```
# 3. Static Method (Independent utility method, uses neither 'self' nor 'cls')
```

```
@staticmethod
def is_passing(marks):
    return marks >= 40
```

```
# Main Execution
```

```
print("=== Student Enrolment No: 92600565003 ===")
print("=== Name: Lakshyadeepsinh ===\n")
```

```
# Creating an instance
```

```
s1 = Student("Lakshyadeepsinh", 85)
```

```
# 1. Calling Instance Method
```

```
print("--- Instance Method ---")
s1.display_details()
```

```
# 2. Calling Class Method
```

```
print("\n--- Class Method ---")
Student.get_college()
```

```
# 3. Calling Static Method
```

```
print("\n--- Static Method ---")
result = Student.is_passing(s1.marks)
print(f"Is Student Passing? {result}")
```

Output:

```
===== RESTART: D:\LV\Msc_92600565003\unit2\10.py ==
M.sc. (CS&CL)Semester 1
Enrollment No:92600565003
=== Student Enrolment No: 92600565003 ===
=== Name: Lakshyadeepsinh ===

--- Instance Method ---
Student Name: Lakshyadeepsinh
Marks: 85

--- Class Method ---
College Name: Marwadi University

--- Static Method ---
Is Student Passing? True
```

Conclusion:

In this practical, the distinctions between Instance Methods, Class Methods (@classmethod), and Static Methods (@staticmethod) were successfully demonstrated based on how they access class variables and instance attributes.

Practical No. 20

Objective: Write a Python program to show method overloading by adding two numbers and three numbers using the same method name.

Source Code:

```
print("M.sc. (CS&CL)Semester 1")
print("Enrollment No:92600565003")
```

```
class Calculator:
    def add(self, a, b, c=None):
        if c is not None:
            return a + b + c
        else:
            return a + b
```

```
print(" Student Enrolment No: 92600565003 ")
```

```
print(" Name: Lakshyadeepsinh \n")

calc = Calculator()

# Adding two numbers using add()
sum_two = calc.add(10, 20)
print("Sum of two numbers (10 + 20):", sum_two)

# Adding three numbers using the same add() method name
sum_three = calc.add(10, 20, 30)
print("Sum of three numbers (10 + 20 + 30):", sum_three)
```

Output:

```
===== RESTART: D:/LV/Msc_92600565003/unit2/20.p
M.sc. (CS&CL)Semester 1
Enrollment No:92600565003
Student Enrolment No: 92600565003
Name: Lakshyadeepsinh

Sum of two numbers (10 + 20): 30
Sum of three numbers (10 + 20 + 30): 60
```

Conclusion:

This practical demonstrated method overloading in Python. By using default arguments within a single method, the program successfully handled addition for both two and three numbers seamlessly.

Practical No. 21

Objective: Write a Python program to show method overriding using a parent class Animal and a child class Dog.

Source Code:

```
print("M.sc. (CS&CL)Semester 1")
print("Enrollment No:92600565003")

# Parent Class
class Animal:
    def speak(self):
        print("Animal makes a generic sound.")

# Child Class overriding the speak method
class Dog(Animal):
```

```
def speak(self):
    print("Dog barks: Woof Woof!")

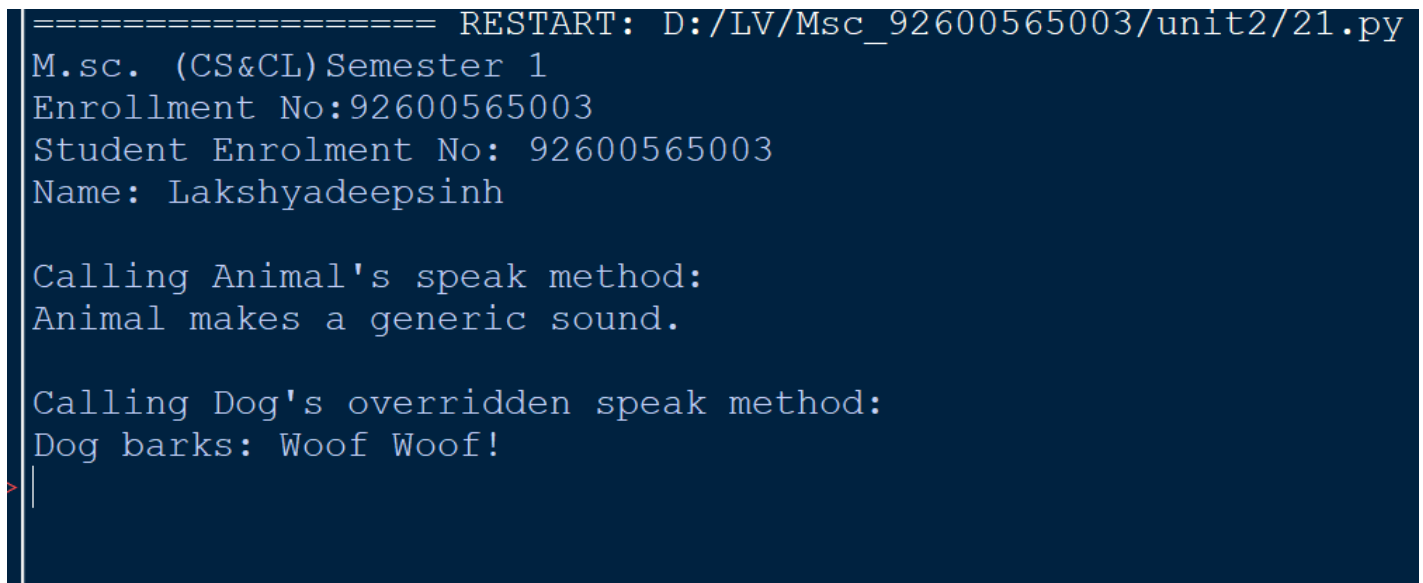
# Main Execution
print("Student Enrolment No: 92600565003 ")
print("Name: Lakshyadeepsinh \n")

# Object of Parent Class
generic_animal = Animal()
print("Calling Animal's speak method:")
generic_animal.speak()

print()

# Object of Child Class
dog = Dog()
print("Calling Dog's overridden speak method:")
dog.speak()
```

Output:



```
===== RESTART: D:/LV/Msc_92600565003/unit2/21.py
M.sc. (CS&CL) Semester 1
Enrollment No:92600565003
Student Enrolment No: 92600565003
Name: Lakshyadeepsinh

Calling Animal's speak method:
Animal makes a generic sound.

Calling Dog's overridden speak method:
Dog barks: Woof Woof!
> |
```

Conclusion:

In this practical, runtime polymorphism via method overriding was implemented. The child class (Dog) successfully redefined and replaced the implementation of a method inherited from the parent class (Animal).